

NYCU DL Lab4 Conditional VAE for Video Prediction

313551073 顏琦恩

I. Derivate conditional VAE formula

To learn a conditional distribution $p(X|C)$, we hope to maximize the marginal distribution $p(X|Z, C; \theta)$, which means to maximize the log-likelihood

$$\log p(X|Z, C; \theta) = \log p(X, Z, C; \theta) - \log p(Z|X, C; \theta)$$

we can rewrite the equation to the following term by applying an arbitrary distribution $q(Z|C)$ on both sides

$$\begin{aligned} & \int q(Z|C) \log p(X|Z, C; \theta) dZ \\ &= \int q(Z|C) \log p(X, Z, C; \theta) dZ - \int q(Z|C) \log p(Z|X, C; \theta) dZ \\ &= \int q(Z|C) \log p(X, Z, C; \theta) dZ - \int q(Z|C) \log q(Z|C) dZ \\ & \quad + \int q(Z|C) \log q(Z|C) dZ - \int q(Z|C) \log p(Z|X, C; \theta) dZ \\ &= \mathcal{L}(X, C, q, \theta) + KL(q(Z|C) || p(Z|X, C; \theta)) \\ \mathcal{L}(X, C, q, \theta) &= \int q(Z|C) \log p(X|Z, C; \theta) dZ - KL(q(Z|C) || p(Z|X, C; \theta)) \\ &\text{then we can rewrite the equation into the following term:} \\ \mathcal{L}(X, C, q, \theta) &= \mathbb{E}_{Z \sim q(Z|X, C; \theta)} \log p(X|Z, C; \theta) - KL(q(Z|X, C; \theta) || p(Z|C)) \end{aligned}$$

II. Introduction

In this lab, I use a Variational Autoencoder (VAE) to predict short videos. By inputting several pose images and a reference image, we can generate a short video with the man in the reference image doing the same poses as the input pose images. With the implementation of the training and testing process and several training methods such as kl annealing and teacher forcing strategy, we can improve the quality of produced video.

III. Implementation details

A. How do you write your training/testing protocol

In my training section, I applied the teacher forcing strategy to decide whether to use the ground truth to be the input of the frame encoder or not. Also,

After training, I use validation datasets to check the PSNR of each epoch to save the better checkpoint. In the validation part, I also applied the same method because the PSNR method can only compute 1 sequence at the same time.

In the testing section, I predict sequence batch by batch for a shorter processing time.

```
152 | # python Trainer.py --DR ../LAB4_Dataset --save_root ./saved_models
153 | def training_one_step(self, img, label, adapt_TeacherForcing):
154 |     # TODO
155 |     # (B, seq, C, H, W)
156 |     batch_size = img.shape[0]
157 |     beta = self.kl_annealing.get_beta()
158 |
159 |     kl_loss = 0.0
160 |     mse_loss = 0.0
161 |     img = img.permute(1, 0, 2, 3, 4) # change tensor into (seq, B, C, H, W)
162 |     label = label.permute(1, 0, 2, 3, 4) # change tensor into (seq, B, C, H, W)
163 |
164 |     decoded_frame = [img[0]]
165 |     for i in range(1, self.args.train_vi_len):
166 |         if adapt_TeacherForcing:
167 |             prev_x = img[i-1]
168 |         else:
169 |             prev_x = decoded_frame[i-1]
170 |
171 |         encoded_prev_x = self.frame_transformation(prev_x)
172 |         encoded_p = self.label_transformation(label[i])
173 |
174 |         encoded_x = self.frame_transformation(img[i])
175 |         z, mu, logvar = self.Gaussian_Predictor(encoded_x, encoded_p)
```

```

176
177         x_hat = self.Generator(self.Decoder_Fusion(encoded_prev_x, encoded_p, z))
178         decoded_frame.append(x_hat)
179
180         kl_loss += kl_criterion(mu, logvar, batch_size)
181         mse_loss += self.mse_criterion(x_hat, img[i])
182
183         loss = mse_loss + beta * kl_loss # seq loss
184
185         self.optim.zero_grad()
186         loss.backward()
187         self.optimizer_step()
188
189         return loss / batch_size # avg seq loss

```

Fig. 1 Details of training code

```

191 # python Trainer.py --DR ../LAB4 Dataset --save_root ./saved_models --ckpt_path ./saved_models/epoch=0.ckpt --test
192 def val_one_step(self, img, label):
193     # TODO
194     batch_size = img.shape[0]
195     batch_psnr = []
196     beta = self.kl_annealing.get_beta()
197
198     kl_loss = 0.0
199     mse_loss = 0.0
200     img = img.permute(1, 0, 2, 3, 4) # change tensor into (seq, B, C, H, W)
201     label = label.permute(1, 0, 2, 3, 4) # change tensor into (seq, B, C, H, W)
202
203     decoded_frame = [img[0]]
204     for i in range(1, self.args.val_vi_len):
205         encoded_prev_x = self.frame_transformation(decoded_frame[i-1])
206         encoded_p = self.label_transformation(label[i])
207
208         encoded_x = self.frame_transformation(img[i])
209         z, mu, logvar = self.Gaussian_Predictor(encoded_x, encoded_p)
210
211         x_hat = self.Generator(self.Decoder_Fusion(encoded_prev_x, encoded_p, z))
212         decoded_frame.append(x_hat)
213
214         kl_loss += kl_criterion(mu, logvar, batch_size)
215         mse_loss += self.mse_criterion(x_hat, img[i])
216         batch_psnr.append(Generate_PSNR(x_hat, img[i]).item())
217
218     loss = mse_loss + beta * kl_loss # seq loss
219
220     return loss / batch_size, batch_psnr

```

Fig. 2 Details of validation code

```

125 # TODO
126 for i in range(1, self.args.val_vi_len):
127     encoded_x = self.frame_transformation(decoded_frame_list[i-1].to(self.args.device))
128     encoded_p = self.label_transformation(label[i]).to(self.args.device)
129
130     z, mu, logvar = self.Gaussian_Predictor(encoded_x, encoded_p) # get shape of z
131     z = torch.randn_like(z) # sample from normal distribution
132
133     x_hat = self.Generator(self.Decoder_Fusion(encoded_x, encoded_p, z))
134     decoded_frame_list.append(x_hat.cpu())
135     label_list.append(encoded_p.cpu())

```

Fig. 3 Details of testing code

B. How do you implement reparameterization tricks

From the paper Auto-Encoding Variational Bayes, we can see an example of implementing the reparameterization trick on page 5, so I just applied the formula in the paper to implement the reparameterization trick, but since the input of sigma is a log variance, I use the exponential to get its standard deviation. The following figures show the formula in the paper and my code of reparameterization.

$$\mathbf{z}^{(i,l)} = \boldsymbol{\mu}^{(i)} + \boldsymbol{\sigma}^{(i)} \odot \boldsymbol{\epsilon}^{(l)} \quad \text{and} \quad \boldsymbol{\epsilon}^{(l)} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$$

Fig. 4 Formula of reparameterization trick in the paper

```
79     def reparameterize(self, mu, logvar):
80         # TODO
81         std = torch.exp(0.5*logvar)
82         eps = torch.randn_like(std)
83         return mu + eps*std
```

Fig. 5 Detail of reparameterization trick

C. How do you set your teacher forcing strategy

I applied the teacher forcing strategy in my training section. It is on if the random number is smaller than the initial teacher forcing ratio (tfr), and off otherwise. I used a linear function to determine the teacher forcing ratio, which is: $tfr = tfr - d_step$. The d_step represents the decay step of tfr, which is set to be 0.1 here. The tfr starts decaying from the 10th epoch beginning at a value of 1.0. When the teacher forcing strategy is on, I used the given image as the previous frame to predict the next frame, on the contrary, I used the predicted frame as the previous frame.

```
265     def teacher_forcing_ratio_update(self):
266         # TODO
267         if self.current_epoch >= self.tfr_sde:
268             self.tfr -= self.tfr_d_step
269             self.tfr = max(0, self.tfr)
```

Fig. 6 Detail of the updating of the teacher forcing strategy

```

def training_one_step(self, img, label, adapt_TeacherForcing):
    # TODO
    # (B, seq, C, H, W)
    batch_size = img.shape[0]
    beta = self.kl_annealing.get_beta()

    kl_loss = 0.0
    mse_loss = 0.0
    img = img.permute(1, 0, 2, 3, 4) # change tensor into (seq, B, C, H, W)
    label = label.permute(1, 0, 2, 3, 4) # change tensor into (seq, B, C, H, W)

    decoded_frame = [img[0]]
    for i in range(1, self.args.train_vi_len):
        if adapt_TeacherForcing:
            prev_x = img[i-1]
        else:
            prev_x = decoded_frame[i-1]

        encoded_prev_x = self.frame_transformation(prev_x)
        encoded_p = self.label_transformation(label[i])

```

Fig. 7 Implementation of the teacher forcing strategy in the training section

D. How do you set your kl annealing ratio

For the kl annealing ratio, I referenced from the paper Cyclical Annealing Schedule: A Simple Approach to Mitigating KL Vanishing.

```

36 class kl_annealing():
37     def __init__(self, args, current_epoch=0):
38         # TODO
39         self.current_epoch = current_epoch
40
41         if args.kl_anneal_type == 'cyclical':
42             self.beta = self.frange_cycle_linear(args.num_epoch, 0.0, 1.0, args.kl_anneal_cycle, args.kl_anneal_ratio)
43         elif args.kl_anneal_type == 'Monotonic':
44             self.beta = self.frange_cycle_linear(args.num_epoch, 0.0, 1.0, 1, args.kl_anneal_ratio)
45         else:
46             self.beta = np.ones(args.num_epoch)
47
48     def update(self):
49         # TODO
50         self.current_epoch += 1
51
52     def get_beta(self):
53         # TODO
54         return self.beta[self.current_epoch]
55
56     def frange_cycle_linear(self, n_iter, start=0.0, stop=1.0, n_cycle=1, ratio=1):
57         # TODO refer to https://github.com/haofuml/cyclical\_annealing/tree/master
58         beta = np.ones(n_iter)
59         period = n_iter/n_cycle # 每經過period個iteration, beta從0開始算
60         step = (stop-start)/(period*ratio) # beta每次增加的量
61
62         for c in range(n_cycle):
63             v, i = start, 0
64             while v <= stop and (int(i+c*period) < n_iter):
65                 beta[int(i+c*period)] = v
66                 v += step
67                 i += 1
68         return beta

```

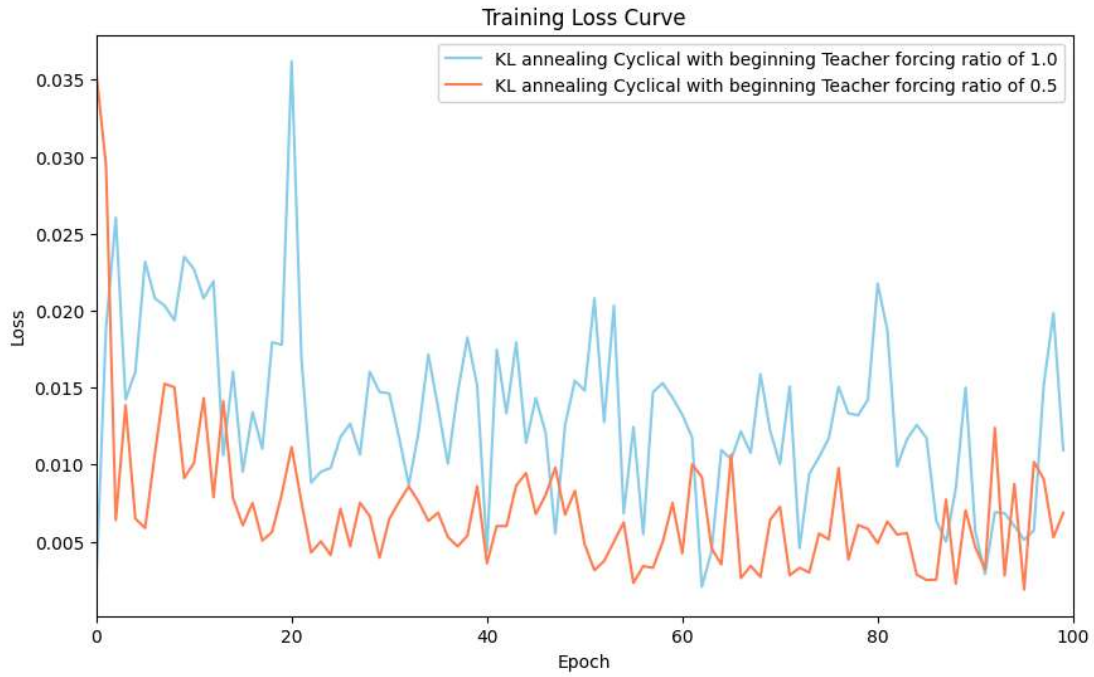
Fig. 8 Implementation of the kl annealing

IV. Analysis & Discussion

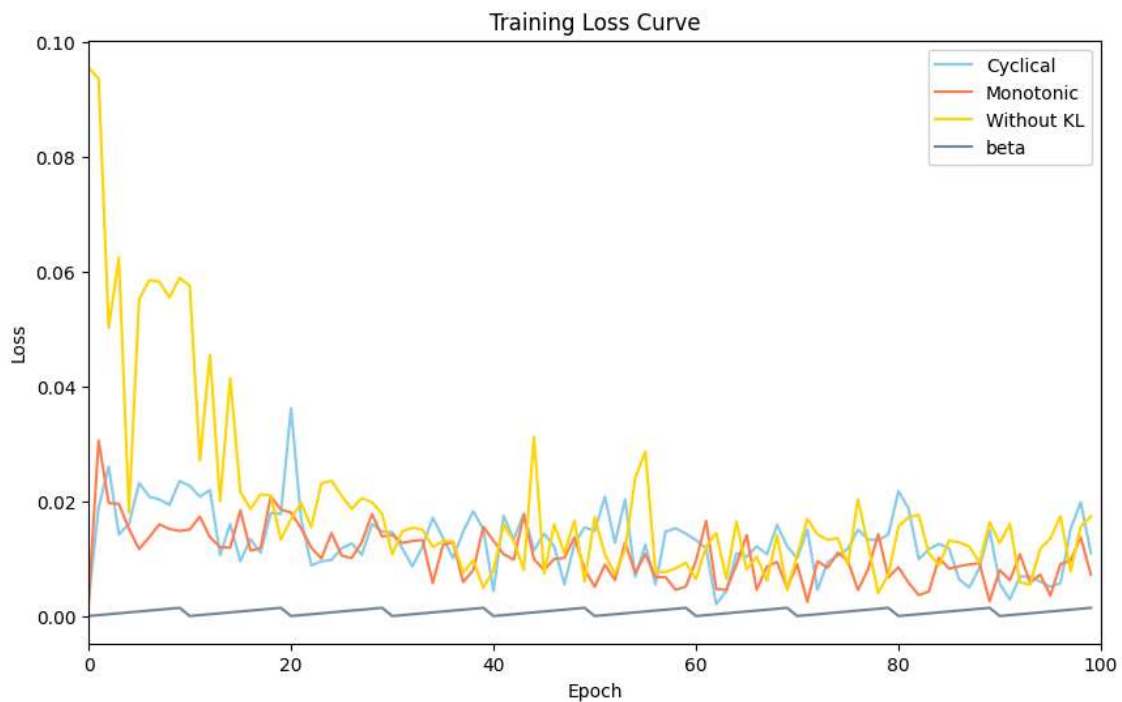
A. Plot Teacher forcing ratio

a. Analysis & compare with the loss curve

This experiment is using Cyclical annealing with the teacher forcing strategy beginning with the teacher forcing ratio of 1.0 and 0.5, to the end of 0.0. We can observe that using a lower beginning ratio of teacher forcing leads to a more stable loss curve in this lab.

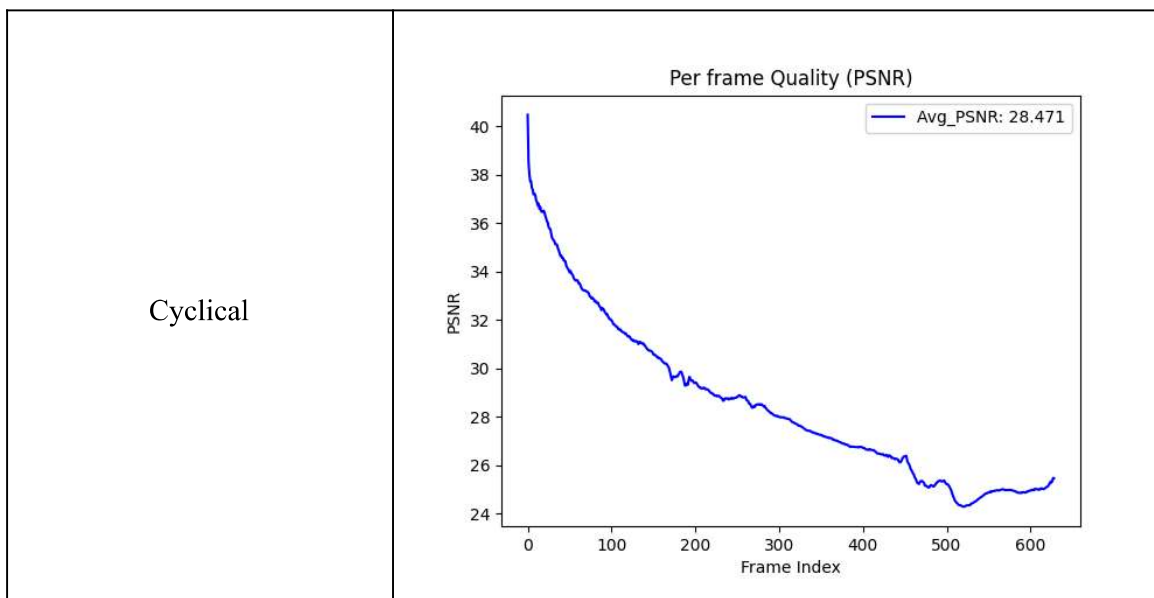


B. Plot the loss curve while training with different settings. Analyze the difference between them

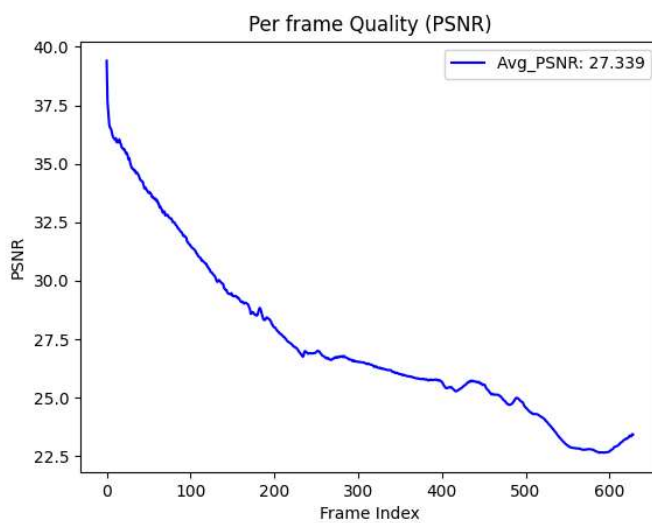


This experiment is using the teacher forcing strategy beginning with the teacher forcing ratio of 1.0 to the end of 0.0. We can see that although different KL annealing strategies have a small impact on the loss curve, the loss curve without KL is more unstable than other strategies, and the Monotonic strategy has the best performance on this task.

C. Plot the PSNR-per frame diagram in validation dataset



Monotonic



Without kl annealing

